Decision-Rights Allocation: A Governance-Centric Perspective on LLM-Enabled Agentic Systems

Oliver Zhou Lily Li

LLM-enabled agentic systems are increasingly deployed as workflow components that retrieve information, reason over context, call tools, coordinate with other agents, and execute actions on behalf of humans. Existing conceptual work has usefully classified such systems by external behavior, architectural components, cognitive design patterns, autonomy levels, or multi-agent governance forms. These frameworks, however, often leave implicit a design choice that becomes central once stochastic language models are connected to tools and organizational processes: who has the right to make each consequential decision inside the workflow? This paper proposes decision-rights allocation as a compact internal-mechanism perspective for understanding and designing LLM-enabled agentic systems. We define an agentic system as an allocation of decision rights among three parties: human actors, deterministic components, and stochastic LLM components. This distinction is especially important because LLMs differ from conventional deterministic automation: their outputs arise from probabilistic language generation, can drift or hallucinate, and can become operationally consequential when connected to tools, memory, and external workflows. The framework interprets common design patterns—including retrieval-augmented generation, episodic memory, human-in-the-loop validation, domain-expert knowledge codification, provenance tracing, reinforcement learning, routing, schema validation, and ontology-derived semantic context—as mechanisms that shift decision rights across these three parties. We further propose a simple optimization model in which the desirable allocation depends on task risk, adaptability needs, scalability pressure, auditability requirements, deterministic codifiability, human capacity, model reliability, verification availability, and the cost of errors at scale. The contribution is a unifying vocabulary for comparing agentic-system designs and a first-step analytical model for deciding when LLMs

should decide, when deterministic infrastructure should constrain them, and when humans should retain or regain control.

Introduction

LLM-enabled agentic systems have moved quickly from demonstrations to applied workflow designs. Recent examples include autonomous anomaly management in complex systems (Barenji and Khoshgoftar 2025), cybersecurity operations (Lazer et al. 2026), business-process management (Dumas, Milani, and Chapela-Campa 2026), mobile-network RAN optimization (Pellejero et al. 2025), software testing (Naqvi, Baqar, and Mohammad 2026; Hariharan 2025), education (Yu et al. 2025), agricultural recommendation (Yamaguchi et al. 2025), engineering knowledge reuse (Son et al. 2026), and credit decisioning (Kubam 2025). Across these settings, the relevant system is no longer only a language model that produces a text response. It is a socio-technical workflow in which LLMs interpret requests, select evidence, plan steps, call tools, write code, update memory, route subtasks, ask for human input, and sometimes trigger actions in external systems.

The theoretical and conceptual literature on such systems has also developed, but it remains fragmented. Some work studies LLM agents as systems composed of perception, reasoning and planning, memory, and execution components (Lamo Castrillo et al. 2025; L. Wang et al. 2023). Some work classifies agentic AI into levels such as LLM agents, agentic AI, and agentic communities (Milosevic and Rabhi 2026). Some work analyzes recurring cognitive design patterns in agentic LLM systems, such as observe–decide–act, episodic memory, commitment, reconsideration, and knowledge compilation (Wray, Kirk, and Laird 2025). Other work proposes ordinal dimensions of agenticness, such as knowledge scope, perception, reasoning, interactivity, operation, contextualization, self-improvement, and normative alignment (Wissuchek and Zschech 2025). These frameworks are valuable because they help researchers describe what agentic systems contain and what behaviors they display.

However, a central design question remains under-specified: who has the right to decide? When a system retrieves documents, is the LLM deciding what evidence matters, or is a deterministic retriever constraining the evidence space? When an agent proposes a credit decision, a test case, a clinical-trial match, or an anomaly response, is the final decision made by the LLM, a rule engine, a human reviewer, or some combination of the three? When an agent learns from feedback or writes to memory, who decides what becomes persistent knowledge? These questions are not merely implementation details. They determine reliability, accountability, scalability, transparency, cost, and organizational control.

This paper proposes decision-rights allocation as a simple perspective for understanding LLM-enabled agentic systems. We argue that most design differences can be interpreted as different

allocations of decision rights among three parties: deterministic components, stochastic LLM components, and human actors. Deterministic components include rules, schemas, retrieval algorithms, databases, validators, routers, calculators, workflow engines, formal ontologies, and other bounded control surfaces. Stochastic LLM components include generative models used for interpretation, reasoning, planning, summarization, action proposal, and natural-language interaction. Human actors include users, operators, domain experts, reviewers, auditors, managers, and affected stakeholders.

The perspective is intentionally compact. It does not replace component-level taxonomies, autonomy scales, cognitive architectures, or system-theoretic governance frameworks. Instead, it provides an internal-mechanism lens that asks, for every consequential step in an agentic workflow, which party holds goal-framing, information-selection, reasoning, action-selection, validation, and memory-update rights. Under this lens, many common design patterns become rights-shifting mechanisms. Retrieval-augmented generation can reduce the LLM's right to invent evidence by giving retrieval infrastructure the right to select or constrain context. Human-in-the-loop validation gives humans veto or approval rights. Domain-expert knowledge codification shifts recurring expert judgment from humans into reusable protocols that guide or constrain LLM outputs. Provenance tracing creates inspectability and contestability, thereby strengthening ex post human and deterministic governance. Episodic memory, reinforcement learning, routing, schema validation, and ontology-derived semantic context similarly shift rights over what the agent recalls, how it adapts, which components it uses, which actions are valid, and which concepts it is allowed to use.

The paper makes three contributions. First, it reframes LLM-enabled agentic systems as allocations of decision rights among deterministic, stochastic, and human parties. Second, it provides a matrix that interprets common agentic-AI design patterns according to how they shift decision rights. Third, it proposes a first-step analytical model for choosing the desirable allocation of rights as a function of task risk, adaptability, scalability, transparency, codifiability, human capacity, model reliability, and verification availability. The goal is not to provide a final quantitative theory, but to offer a shared vocabulary for comparing designs and for asking why a particular system grants too much or too little decision authority to the LLM.

Existing conceptual foundations for agentic systems

Fragmented definitions and higher-level frameworks

A wide range of terms is now used to describe AI systems that act on behalf of humans: AI agents (Sapkota, Roumeliotis, and Karkee 2025), LLM-based agents (Lamo Castrillo et al. 2025; Milosevic and Rabhi 2026), prompt-driven generative AI (Lazer et al. 2026), agentic

workflow systems (Kamalov et al. 2025; Fiorini et al. 2025), agents augmented with codified expert knowledge (Uulu et al. 2026), agentic AI (Sapkota, Roumeliotis, and Karkee 2025; Barenji and Khoshgoftar 2025; Dumas, Milani, and Chapela-Campa 2026; Lazer et al. 2026), agentic systems (Hu, Lu, and Clune 2024), agentic LLM systems (Wray, Kirk, and Laird 2025), and agentic communities (Milosevic and Rabhi 2026). This terminology reflects a real expansion in system capability, but it also creates difficulty for comparison. A chatbot with tool access, a multi-agent workflow, a workflow recommender with episodic memory, and a governed enterprise agent community may all be called agentic, even though the locus of decision-making is very different in each case.

Several recent frameworks attempt to reduce this ambiguity by classifying observable function or outcome. Wissuchek and Zschech (2025) conceptualizes agenticness as a spectrum of functional behavior across eight ordinal dimensions: knowledge scope, perception, reasoning, interactivity, operation, contextualization, self-improvement, and normative alignment. The value of this approach is that it avoids treating agency as binary. It makes possible a graded comparison between systems that have different degrees of cognitive and environmental agency. Yet, because the dimensions describe what the system appears able to do, they do not directly answer whether the LLM, the human, or deterministic infrastructure should be allowed to exercise those capabilities in a particular task.

Another line of work imports ideas from cognitive architectures and classical agent architectures. Wray, Kirk, and Laird (2025) argues that agentic LLM systems can be compared through cognitive design patterns such as observe–decide–act, episodic memory, commitment, reconsideration, and knowledge compilation. This provides a bridge between current LLM systems and older architectures such as BDI, ACT-R, and Soar (Laird 2022). This line of work is especially relevant to decision rights because many consequential decisions are made inside the cognitive module of an agentic system: what to observe, which plan to form, what to remember, and whether to reconsider. However, a cognitive pattern is still not the same as a rights allocation. The same observe–decide–act pattern can be implemented with LLM-dominant planning, deterministic policy-constrained planning, or human-approved planning.

A third line of work focuses on multi-agent and enterprise settings. Milosevic and Rabhi (2026) distinguishes LLM agents, agentic AI, and agentic communities, emphasizing that production-grade systems require roles, protocols, governance, and accountability. AutoGen-style systems similarly show how LLM applications can be built as conversations among customizable agents, tools, and humans (Wu et al. 2023). This work moves closer to decision rights because it recognizes formal roles, obligations, verification, and accountability. However, it generally classifies the architecture or coordination pattern rather than reducing the design problem to a comparable allocation among humans, deterministic infrastructure, and stochastic LLMs.

Business-process management provides a related but more domain-specific example. Dumas, Milani, and Chapela-Campa (2026) argues that agentic AI extends business-process management from predefined automation toward autonomous performers and that verification-centric

process design becomes necessary when autonomous decision-making enters organizational workflows. This is not simply another high-level taxonomy of agentic systems. It is evidence that, in one organizational domain, the practical question is how much authority to transfer from predefined process logic and human process owners to LLM-enabled performers. In the language of this paper, the BPM transition can be interpreted as a shift in decision rights rather than merely a shift in process technology.

Autonomy, oversight, and why the existing foundations are not enough

Existing frameworks therefore provide necessary but incomplete foundations. Component taxonomies explain what modules exist. Cognitive design patterns explain what functions recur across intelligent systems. Agenticness typologies explain how much agency a system appears to exhibit. Enterprise-governance frameworks explain how multiple agents and humans coordinate. Autonomy scales explain how independently a system can operate. But practical system design requires an additional question: at each workflow step, who is empowered to choose, constrain, validate, and remember?

This question matters because the same architecture can imply different governance properties depending on how decision rights are allocated. A retrieval-augmented system may be conservative if deterministic retrieval selects a curated evidence set and the LLM only summarizes it. The same system may be risky if the LLM writes its own search query, selects among conflicting results, and acts on retrieved content without validation. A memory system may improve continuity if humans approve persistent memories; it may amplify errors if the LLM writes unverified conclusions into long-term memory. A multi-agent workflow may improve specialization if deterministic orchestration allocates subtasks based on verified competence; it may create opaque diffusion of responsibility if LLM agents delegate decisions to one another without audit trails.

For this reason, decision-rights allocation is not an alternative name for autonomy. Autonomy describes the extent to which a system can operate without external intervention. Decision-rights allocation describes which party has authority over specific decisions. A system can be highly automated yet grant final approval rights to humans. Another system can appear human-in-the-loop while still granting the LLM de facto rights over evidence selection, framing, and interpretation. The distinction is important for agentic systems because control can be lost not only at the final action step but also upstream through context construction, tool selection, memory updates, and default recommendations.

Recent autonomy-oriented work reinforces this distinction. Autonomous multi-agent architectures can be assessed jointly through autonomy levels, alignment levels, and architectural viewpoints rather than by a single flat autonomy score (Händler 2023). AI-agent autonomy can also be treated as a deliberate design decision separable from capability and operational

environment, with user-centered levels such as operator, collaborator, consultant, approver, and observer (Feng, McDonald, and Zhang 2025). Other work proposes code-inspectable autonomy attributes (Cihon et al. 2025), data-agent autonomy levels that track the progressive transfer of dominance and responsibility (Y. Zhu et al. 2025), and observation-based measures of autonomy for in-the-wild systems (Pittman 2024). These contributions are useful, but they still need to be translated into a finer question: which rights are transferred, at which decision nodes, under which safeguards?

The same point applies to human oversight. Risk-proportionate oversight frameworks distinguish Human-in-Command, Human-in-the-Loop, and Human-on-the-Loop arrangements according to model influence and decision consequence (Kandikatla and Radeljic 2025). Decision-rights allocation provides a mechanism-level interpretation of these categories. Human-in-Command is not merely more oversight than Human-on-the-Loop; it means humans retain goal-framing and ultimate validation rights. Human-in-the-Loop may mean humans hold validation rights but not information-selection rights. Human-on-the-Loop may mean deterministic monitoring holds routine validation rights while humans retain escalation rights. Thus, oversight labels become more precise when decomposed into rights.

Why LLM-enabled agentic systems require a distinct design lens

Traditional software and many conventional machine-learning systems also allocate decision rights. Rule-based systems allocate rights to deterministic code written by developers. Classical machine-learning systems allocate some rights to trained statistical models while often preserving deterministic preprocessing, scoring thresholds, and post-processing rules. Human workflow systems allocate rights to human operators, managers, and reviewers. LLM-enabled agentic systems differ because they combine probabilistic language generation, natural-language reasoning, tool use, memory, and multi-step workflow execution in a single operational loop.

The first distinctive feature is stochastic and under-specified behavior. LLMs generate text autoregressively from estimated conditional distributions. Small token-level probability errors can compound into later factual or coherence drift, and uncertainty may arise from input ambiguity, knowledge gaps, and decoding randomness (Kiprono 2025; Taparia et al. 2026). Hallucination can be framed as the risk that model output contains intrinsic contradictions against a source or extrinsic additions unsupported by available evidence; for open-ended general-purpose LLMs, complete elimination of hallucination is unlikely, so detection and mitigation remain necessary (Gumaan 2025). This stochasticity distinguishes LLM-enabled agentic systems from autonomous but deterministic systems. A deterministic controller may

make a wrong decision, but its policy can often be replayed as fixed code or a fixed rule table. An LLM-generated decision is instead produced through a high-dimensional learned distribution whose failure modes may change with prompt, context, sampling setting, model version, and surrounding tools.

A further nuance is that deterministic decoding does not turn an LLM into a deterministic component in the governance sense used here. Greedy decoding can collapse a conditional output distribution into a single trajectory, but this may hide uncertainty, alternative valid solutions, and harmful completions in non-argmax regions of the model's output distribution (Joshi et al. 2026). Deterministic decoding is useful for debugging, regression testing, and replay, but it does not make the model's internal decision process equivalent to a rule engine, schema validator, or calculator. In decision-right terms, the LLM still holds stochastic semantic judgment rights even when the sampling procedure is made reproducible.

The second distinctive feature is that LLM agents translate language into action. In a simple chatbot, hallucination is mainly an information-quality problem. In an agentic system, hallucination or misinterpretation can become a workflow problem because the system may call tools, execute code, update records, contact users, or trigger external processes. Cao et al. (2026) notes that stochastic LLM outputs create integration risk when they must control deterministic backend systems that require schema-conformant inputs. Contract-driven development, policy projection, and constrained action spaces are therefore not merely engineering conveniences; they are mechanisms for moving action-selection and execution rights away from unconstrained generation and into deterministic control surfaces.

The third distinctive feature is persistent context. Agentic systems increasingly use retrieval, episodic memory, graph-memoized reasoning, workflow databases, and reflective memory consolidation (Fiorini et al. 2025; Singh 2025; J. Wang 2025; Cao et al. 2026). Memory can improve reuse and consistency, but it also changes the temporal structure of decision rights. A false conclusion stored today can influence tomorrow's recommendation. A workflow trace retrieved from a previous case can become a precedent. A graph-memoized reasoning subgraph can reduce cost but may encode outdated or context-inappropriate assumptions. Thus, the right to write, retrieve, revise, and delete memory becomes a central governance issue.

The fourth distinctive feature is the need for system-level interpretability. Agentic behavior emerges from coordinated planning, memory, tool use, routing, and multi-agent interaction rather than from a single static prediction. J. Zhu et al. (2026) argues that post-hoc explainability methods designed for static input-output models face limits in such settings. Similarly, McGee et al. (2025) suggests that ontology-derived semantic context and Toulmin-style justification loops can make agentic decisions more inspectable through explicit claims, evidence, warrants, backing, rebuttals, and qualifiers. These methods can be interpreted as shifting some reasoning and justification rights from opaque LLM generation toward inspectable structures that humans and deterministic systems can evaluate.

The fifth distinctive feature is the organizational role of human expertise and feedback. In many high-value domains, relevant knowledge is tacit, evolving, or situated. L. Zhang (2026) frames domain-agent development as continuous co-development in which practitioner corrections and recurring requests become durable evaluation rules, error libraries, and skills. G. Zhang (2025) argues that knowledge protocols can convert expert knowledge into executable LLM guidance, including workflows, decision trees, logical dependencies, and heuristic strategies. End users also matter because their feedback, complaints, corrections, and acceptance behavior can become signals for future model behavior or workflow redesign. These approaches show that the design problem is not simply whether to automate a human task. It is how to transform selected human decision rights into codified guidance while preserving human rights over ambiguous, high-stakes, or evolving judgments.

The sixth distinctive feature is the scalability paradox. Agentic AI is often attractive because it can scale: a single workflow can process many cases, operate continuously, and reduce human bottlenecks. But output generation that scales easily can also scale errors, review burdens, incident response, downstream remediation, and accountability costs. LLM rights are therefore risky when outputs are expensive at scale not only because model calls may be costly, but because a small error rate can create large aggregate rework, liability, or user harm when multiplied across many autonomous actions. Scalability strengthens the case for automation, but it also strengthens the case for deterministic verification, monitoring, and escalation design.

Decision-rights allocation framework

Three parties and component boundaries

We define an LLM-enabled agentic system as a workflow in which decision rights are allocated among three parties.

The first party is the **human**. Human actors include end users, domain experts, workflow owners, compliance officers, reviewers, auditors, managers, and affected stakeholders. Humans can provide goals, tacit knowledge, preferences, ethical judgment, contextual interpretation, validation, escalation decisions, accountability, and feedback. Human decision rights are valuable when tasks require responsibility, contextual judgment, legitimacy, and interpretation of ambiguous social or organizational constraints. They are costly when they create bottlenecks, slow response, introduce inconsistency, or make scaling difficult.

The second party is the **deterministic component**. Deterministic components include conventional software, rule engines, schemas, calculators, databases, retrievers, bounded routers, workflow engines, validators, knowledge graphs, ontologies, and policy constraints. Their

rights are valuable when repeatability, auditability, cost control, formal compliance, and replayability matter. They are limited when the environment is too open-ended, when rules are incomplete, when knowledge changes quickly, or when tacit expertise cannot be fully specified.

The third party is the **stochastic LLM component**. The LLM can interpret natural language, generate plans, summarize evidence, draft outputs, reason over ambiguous context, translate between formats, and adapt to novel cases. LLM decision rights are valuable when tasks require flexibility, semantic understanding, rapid adaptation, and scalable handling of heterogeneous inputs. They are risky when outputs are unreliable, difficult to audit, sensitive to prompt injection, connected to irreversible external actions, or cheap to generate but costly to verify and remediate at scale.

The boundary between deterministic and stochastic components deserves explicit treatment. Prompt templates, prompt engineering, orchestration code, retrieval policies, schema constraints, output validators, and approval gates are not the LLM component even though they shape LLM behavior. They are deterministic or semi-deterministic control infrastructure around the LLM. Conversely, the LLM component should be understood as the model or model-service output at the system boundary. In practice, an external LLM API provider may use hidden deterministic tools, safety filters, retrieval systems, or post-processing steps. If these mechanisms are not observable or controllable by the system designer, the deployed system cannot allocate explicit deterministic rights to them. Their effect appears only as an empirical change in the reliability and behavior of the LLM service. If the provider exposes a tool, retrieval source, validator, or contract as a controllable interface, then that exposed interface can be modeled separately as deterministic infrastructure.

These three parties should not be understood as mutually exclusive modules. A human may create a deterministic protocol that guides the LLM. An LLM may propose a rule that a human later approves. A deterministic router may choose which LLM to call. A human may validate an ontology-derived representation before it is used by the agent. A learned router may behave deterministically at runtime while still being estimated from data; in this framework it counts as deterministic infrastructure only to the extent that its action set, decision rules, confidence thresholds, and logs are bounded and auditable. The point of the framework is to ask which party has effective authority over each decision, not which party appears in the software diagram.

Decision rights within a workflow

A workflow can be decomposed into decision nodes. At each node, six primary types of rights may be allocated:

1. **Goal-framing rights**: who defines the objective, constraints, and acceptable trade-offs?
2. **Information-selection rights**: who decides what evidence, memory, context, or tool output is relevant?
3. **Reasoning rights**: who interprets the evidence and derives intermediate conclusions?
4. **Action-selection rights**: who chooses the next tool call, workflow transition, external action, or final recommendation?
5. **Validation rights**: who approves, rejects, revises, escalates, or audits the output?
6. **Memory-update rights**: who decides what should be stored, reused, revised, or forgotten?

These six categories are intended as primary rights rather than an exhaustive vocabulary of every operational subright. Terms such as representation rights, component-selection rights, execution rights, feedback rights, and context-construction rights can be mapped onto the six categories. For example, representation rights are a form of information-selection and reasoning rights; routing rights are a form of action-selection rights; schema-based execution rights are a constrained form of action-selection and validation rights; user-feedback incorporation is a form of validation and memory-update rights. This mapping keeps the framework compact while allowing applied analyses to use more granular labels.

Let a workflow contain decision nodes indexed by $j = 1, \dots, J$ and right types indexed by $m = 1, \dots, M$. For each pair (j, m), define a decision-right vector

$$r_{jm} = (h_{jm}, d_{jm}, l_{jm}), \quad h_{jm} + d_{jm} + l_{jm} = 1, \quad h_{jm}, d_{jm}, l_{jm} \geq 0,$$

where h_{jm} is the share of effective decision right held by humans, d_{jm} is the share held by deterministic components, and l_{jm} is the share held by the LLM. This is not meant to imply that rights are always literally divisible. It is a measurement abstraction. A step in which the LLM drafts a recommendation but a human must approve it may have high LLM reasoning rights but high human validation rights. A step in which a schema blocks invalid tool calls gives deterministic components high action-selection and validation rights even if the LLM proposes the action.

The full object of design is the rights matrix $\{r_{jm}\}_{j,m}$. The aggregate system profile is a summary, not a substitute for the matrix:

$$R = \sum_{j=1}^{J} \sum_{m=1}^{M} \omega_j \lambda_m r_{jm},$$

where ω_j captures the importance of workflow node j, and λ_m captures the importance of right type m. In low-risk drafting tasks, reasoning and output-generation rights may matter more

than final validation. In regulated credit, medical, cybersecurity, or infrastructure settings, action-selection, validation, and memory-update rights may receive higher weights.

The purpose of the aggregate profile is threefold. First, it provides a compact diagnostic: a system may be globally LLM-dominant even if it has a few human approval gates. Second, it allows high-level comparison across systems after their workflows are aligned to a common node ontology. Third, it can be used inside an optimization model when the full matrix is too detailed for early-stage design. However, the aggregate should always be traceable back to the underlying node-level rights because two systems with the same R may allocate rights very differently across high-risk and low-risk nodes.

Comparison does not require two systems to have identical components. It requires a mapping between decision nodes or task functions. For example, two credit-decision systems can be compared by aligning intake, evidence retrieval, eligibility assessment, explanation drafting, final approval, and record update, even if one uses RAG and the other uses a rules database. After such alignment, a distance between two designs can be represented as

$$\Delta(A, B) = \sum_{j=1}^{J} \sum_{m=1}^{M} \omega_j \lambda_m \| r_{jm}^{A} - r_{jm}^{B} \|_1 .$$

The node weights ω_j and right-type weights λ_m can be set in several ways. In a qualitative paper, equal weights may be sufficient for exposition. In an applied system, ω_j can be estimated from decision frequency, consequence severity, reversibility, regulatory exposure, and historical failure cost. The weights λ_m can be elicited from domain experts, compliance requirements, user-impact analysis, or structured methods such as pairwise comparison. They can also be learned or adjusted from incident logs if the organization has enough operational data. The important point is that weights should represent the local value and risk of rights, not the salience of a component in the architecture diagram.

Extreme points and trade-offs

The three extreme points of the allocation space illustrate the trade-offs.

At the **human-dominant** extreme, humans hold most decision rights. This is often appropriate when the task is rare, high-stakes, normatively sensitive, or heavily dependent on tacit expertise. It maximizes human accountability and contextual judgment, but it limits scalability and responsiveness. Human-dependent anomaly-management pipelines, for example, can constrain responsiveness when timely intervention is required in complex systems (Barenji and Khoshgoftar 2025).

At the **deterministic-dominant** extreme, rules, schemas, workflows, retrievers, validators, and formal knowledge structures hold most decision rights. This can improve reliability, auditability, replayability, and cost control. It is appropriate when the task is stable, codifiable, and measurable. Deterministic validation can also expose unsupported factual claims by translating claims into symbolic checks or knowledge-base queries (Vakharia et al. 2024), and neuro-symbolic pipelines can move parts of reasoning into explicit rules, ontologies, or automated consistency checks (Vsevolodovna and Monti 2025; Garrido-Merchán and Puente 2025). However, deterministic systems can be brittle when behavior evolves, novel-but-valid patterns appear, or sensor noise creates false positives (Barenji and Khoshgoftar 2025). They may also be difficult to build when expert knowledge is tacit or when the relevant context changes faster than rules can be maintained.

At the **LLM-dominant** extreme, the LLM holds broad rights over interpretation, reasoning, planning, and action selection. This can improve adaptability, natural-language usability, and coverage across heterogeneous cases. It is attractive for tasks that are difficult to specify in advance. But it can reduce auditability, increase hallucination risk, create integration errors, and produce outputs that are cheap to generate but expensive to verify. The empirical results of Z. Z. Wang et al. (2025) suggest that LLM agents may be fast and cheap while still producing lower-quality work in many realistic tasks. Repetitive deterministic-prediction tasks may also reveal sequence-level accuracy cliffs as output length grows (Hou et al. 2025), reinforcing the point that LLM-dominant rights should be matched to tasks where semantic flexibility is more valuable than exact repeatability.

Most useful agentic systems are therefore hybrid. The design question is not whether an LLM should be present, but which rights it should hold, which rights should be constrained by deterministic infrastructure, and which rights should remain with humans.

Design patterns as decision-right-shifting mechanisms

Common agentic-system design patterns can be reinterpreted as mechanisms for reallocating decision rights. This section maps several patterns to the rights they primarily affect and connects the six-right framework to related classifications in the literature.

Design pattern	Primary mechanism	Main decision-right shift	Important caveat
Retrieval-augmented generation (RAG)	Retrieves external documents or records before generation	Shifts information-selection rights from the LLM's parametric memory toward retrieval infrastructure	If the LLM writes the query, chooses sources, or resolves conflicts without validation, substantial rights remain with the LLM
Episodic memory	Stores and retrieves prior interactions, workflows, or reasoning traces	Shifts information-selection and memory-update rights toward memory infrastructure	If the LLM writes memories without approval, errors can become persistent precedents
Human-in-the-loop or human-on-the-loop validation	Requires human approval, correction, monitoring, or escalation	Shifts validation and sometimes action-selection rights to humans	Can create bottlenecks; oversight may be superficial if upstream framing is controlled by the LLM
Domain-expert knowledge codification	Converts expert heuristics into protocols, rules, examples, or executable guidance	Shifts recurring reasoning and validation rights from humans into deterministic guidance that constrains the LLM	Codified knowledge can become stale; tacit knowledge may be only partially captured
Provenance tracing and audit logs	Records evidence, intermediate states, tool calls, and decision paths	Strengthens ex post human and deterministic validation rights	Provenance does not itself prevent bad decisions unless linked to validation or rollback mechanisms

Design pattern	Primary mechanism	Main decision-right shift	Important caveat
Reinforcement learning and feedback adaptation	Updates future behavior according to rewards or feedback	Shifts future reasoning, action-selection, and memory-update rights toward the learned policy and reward specification	Poor reward design can over-optimize measurable outcomes while weakening accountability
Ontology-derived semantic context	Uses formal concepts and relationships to structure interpretation	Shifts information-selection and reasoning rights toward deterministic semantic models	Ontologies require maintenance and human validation to avoid institutional amnesia or false structure
Routers and cascades	Selects models, prompts, retrieval methods, precision levels, or tools based on task features	Shifts action-selection rights from ad hoc prompting to deterministic or learned routing	Router errors can silently send tasks to inappropriate components
Schema and contract validation	Requires structured, valid, policy-compliant outputs before execution	Shifts validation and action-selection rights to deterministic interfaces	Valid format does not guarantee correct reasoning or appropriate action
Multi-agent orchestration	Decomposes tasks among specialized agents or roles	Distributes goal-framing, reasoning, action-selection, and validation rights across agents, supervisors, tools, and humans	Without explicit governance, distributed autonomy can obscure responsibility
Agent evaluation and judging	Evaluates outputs, trajectories, artifacts, or intermediate workspaces	Shifts validation rights toward human judges, LLM judges, agent judges, or deterministic test suites	Evaluation rights can be biased or incomplete if judges see only final outputs

This table complements typological work on agenticness. Wissuchek and Zschech (2025) classifies systems by dimensions of cognitive and environmental agency. The table above asks a different question: once a system has a given capability, which party controls that capability in deployment? It also complements decoupled HITL architectures, which treat human approval, escalation, and feedback as an independent component of the agent operating environment (Cheng and Cheng 2026). In rights terms, a decoupled HITL layer is valuable because it makes validation and escalation rights explicit rather than embedding them informally inside each application.

RAG is a simple example. In a purely generative system, the LLM has broad rights over what information it uses and how it justifies an answer. In a RAG system, a retriever can narrow the evidence set and thereby reduce the LLM's right to invent or select facts. However, the shift is incomplete if the LLM controls query formulation, source ranking, or conflict resolution. This explains why RAG design is not only about accuracy; it is about allocating evidence-selection rights.

Episodic memory has a similar structure. Fiorini et al. (2025) describes provenance-backed workflow databases as a basis for episodic memory in agentic workflows. Singh (2025) describes graph-memoized reasoning as persistent graph-structured memory that allows prior reasoning subgraphs to be retrieved and recomposed. These designs shift some reasoning effort away from the LLM at run time, but they also create governance questions: who decides which traces are stored, which traces are retrieved, and when a prior trace is no longer valid?

Human-in-the-loop validation is the most explicit rights-shifting pattern. It gives humans approval, correction, or veto rights. Many applied systems use this pattern because full LLM autonomy remains unreliable in high-stakes or feedback-sensitive tasks. Human oversight appears in software testing (Naqvi, Baqar, and Mohammad 2026; Hariharan 2025), education (Yu et al. 2025), and rice-yield recommendation (Yamaguchi et al. 2025). Yet human-in-the-loop design can be weaker than it appears if the human sees only the final answer and not the evidence, tool calls, rejected alternatives, or uncertainty. Effective human rights require observability and meaningful intervention points.

Domain-expert knowledge codification shifts rights in a different direction. Uulu et al. (2026) argues that codified expert knowledge can improve agent outputs beyond RAG-only code generation, because factual retrieval alone may not encode what makes an output analytically useful. G. Zhang (2025) similarly argues that knowledge protocols can specialize general-purpose models through workflows, decision trees, logical dependencies, and heuristic strategies. In decision-right terms, these methods move repeated expert judgments from direct human intervention into deterministic or semi-deterministic guidance. This can improve scalability while preserving some expert intent.

Provenance tracing, audit logs, and justification loops are often described as explainability tools, but their governance function is broader. They create the conditions for review, contes-

tation, and accountability. McGee et al. (2025) proposes a Toulmin-style justification loop for inspecting agentic decisions through explicit claims, evidence, warrants, backing, rebuttals, and qualifiers. J. Zhu et al. (2026) argues for system-level interpretability because agentic behavior emerges across modules. These mechanisms do not necessarily reduce LLM decision rights at the moment of generation, but they increase human and deterministic rights to audit, reject, revise, and learn from decisions afterward.

Reinforcement learning and feedback adaptation allocate rights over future behavior. In software testing, Hariharan (2025) describes reward dimensions such as effectiveness, coverage, efficiency, compliance, and adaptation. In interaction learning, Klissarov et al. (2026) argues that feedback adaptation should be optimized rather than left to prompt engineering. In decision-right terms, reward functions and feedback mechanisms decide which behaviors become more likely. This shifts some rights from current human operators or prompt designers into the learned policy. The benefits are adaptability and continual improvement; the risks are reward misspecification, hidden drift, and reduced interpretability.

Ontology-derived semantic context shifts representational rights, which are treated here as a subcase of information-selection and reasoning rights. McGee et al. (2025) argues that ontology-derived semantic context can improve LLM response accuracy and coherence in enterprise operations and that human validation of AI-generated knowledge structures can reduce institutional amnesia. The ontology constrains what concepts and relations the system uses, while human validation prevents the ontology from becoming a misleading deterministic artifact. This pattern is especially important when organizations want agents to operate over institutional knowledge rather than over free-form retrieved text.

Routers and cascades allocate rights over component selection. Routing can choose among LLMs, quantized and full-precision variants, tools, direct answering, retrieval, or human escalation. Learned routing work shows that routing can be optimized for quality, cost, latency, and decision regret (Sikeridis, Ramdass, and Pareek 2024; Tsiourvas, Sun, and Perakis 2025; Qian et al. 2025; Y. Li et al. 2026). It can also fail through routing collapse, where a policy increasingly defaults to the strongest and most expensive model even when cheaper models are sufficient (Lai and Ye 2026). In decision-right terms, routers are governance mechanisms because they decide which party or component will exercise downstream reasoning and action-selection rights.

Multi-agent orchestration distributes rights across multiple LLMs, tools, and humans. Frameworks such as AutoGen demonstrate how agent interactions can be programmed through conversations, Python control logic, auto-reply functions, and group chats (Wu et al. 2023). Framework-agnostic representations such as Agent Spec attempt to standardize agent components, control-flow semantics, data-flow semantics, and runtime adapters (Amini et al. 2025). These efforts are important because rights become harder to see when they are spread across agents. A rights-aware multi-agent design should specify not only roles, but also which agent can set goals, select evidence, call tools, approve actions, and write memory.

A model for choosing the desirable allocation

The decision-rights perspective becomes useful when it guides design choices. We propose a simple comparative model. A system design a implies a node-by-right matrix $\{r_{jm}(a)\}_{j,m}$ and an aggregate rights profile

$$R(a) = (H(a), D(a), L(a)), \quad H(a) + D(a) + L(a) = 1.$$

The desirable design maximizes expected system value subject to the rights matrix induced by the architecture, interface, model choice, workflow policy, and oversight design:

$$a^* = \arg\max_a \ \mathbb{E}[V(a)] = B_{scale}(a) + B_{adapt}(a) + B_{quality}(a) - C_{human}(a) - C_{compute}(a) - C_{integration}(a)$$

This formulation is deliberately high-level. It does not assume a universal linear function. Instead, it identifies variables that should change the desirable allocation. The additional term $C_{error\_scale}(a)$ emphasizes that scalable agentic systems can scale not only productivity but also mistakes, remediation cost, compliance exposure, and loss of trust.

Variable	Expected effect on LLM rights	Expected effect on deterministic rights	Expected effect on human rights
Scalability pressure	Increases, if quality and safeguards are acceptable	Increases when tasks are codifiable or checkable	Decreases when human review is bottleneck
Need for adaptability	Increases for open-ended interpretation and planning	Decreases if rules cannot keep up	Increases when adaptation requires tacit judgment
Deterministic codifiability	Decreases	Increases	Decreases after expert knowledge is codified
Task risk and irreversibility	Decreases unless strong safeguards exist	Increases through constraints and validators	Increases through approval and accountability
Auditability and regulatory burden	Decreases for opaque reasoning	Increases through logs, schemas, rules, and provenance	Increases through review and sign-off
LLM task reliability	Increases	May decrease for that subtask	May decrease for routine validation

Variable	Expected effect on LLM rights	Expected effect on deterministic rights	Expected effect on human rights
Verification availability	Increases when outputs can be automatically checked	Increases through validators and tests	Decreases for routine checks but remains for exceptions
Human capacity or cost pressure	Increases for low-risk tasks	Increases for automatable subtasks	Decreases except for escalation
Tacitness of domain knowledge	Increases for elicitation and drafting	Increases after knowledge is codified	Increases for validation and correction
Security and prompt-injection exposure	Decreases for untrusted inputs	Increases through filtering, sandboxing, and least-privilege tools	Increases for high-risk exceptions
Need for user feedback and contestability	Increases for feedback summarization and triage	Increases through feedback schemas and issue trackers	Increases for appeal, correction, and policy revision

The model yields several design propositions.

Proposition 1: LLM decision rights should increase with adaptability needs, but only when verification is available. LLMs are valuable when tasks require semantic flexibility, novel interpretation, or rapid response to changing environments. This is why rule-based anomaly management faces limitations in evolving systems (Barenji and Khoshgoftar 2025). However, adaptability without verification can produce fabricated or misdirected actions. Therefore, high LLM rights are most defensible when paired with deterministic tests, provenance, rollback, human escalation, or other validation mechanisms.

Proposition 2: deterministic decision rights should increase with codifiability, risk, and auditability requirements. When a decision can be represented as a rule, schema, ontology, retrieval policy, router, or validator, deterministic infrastructure can reduce variance and improve auditability. This does not require removing the LLM. The LLM can still interpret inputs or draft outputs, while deterministic components hold rights over admissible evidence, valid actions, arithmetic, compliance checks, routing thresholds, and memory write conditions.

Proposition 3: human decision rights should concentrate at high-leverage points rather than appear everywhere. Human-in-the-loop design is often necessary, but indiscriminate human review weakens scalability and may create rubber-stamping. A better design places humans where their rights are most valuable: goal framing, validation of high-risk actions, adjudication of ambiguous cases, approval of persistent memory, review of user feedback, and

refinement of domain protocols. This interpretation is consistent with human-centered evaluation in optimization workflows, where interpretability, computational efficiency, domain constraints, and tacit preferences are hard to encode in fixed fitness functions (W. Li et al. 2025).

Proposition 4: memory-update rights require stricter governance than one-shot generation rights. A one-time LLM error can be corrected. A stored error can become a future premise. Memory systems therefore require explicit rights over writing, retrieval, versioning, provenance, and deletion. Persistent reasoning reuse should be conditioned on policy constraints such as acyclicity, type compatibility, semantic thresholds, version control, and provenance metadata (Singh 2025).

Proposition 5: the optimal allocation is workflow-specific, not system-wide. A single system may give the LLM high rights in drafting, deterministic components high rights in retrieval and validation, and humans high rights in approval and escalation. Treating an entire system as either autonomous or human-controlled obscures these internal allocations.

Proposition 6: high scalability raises both the value of delegation and the value of control. When a task occurs many times, delegating rights to LLMs or deterministic systems can create large productivity gains. But the same frequency increases the expected cost of systematic errors, biased defaults, memory contamination, and user-harm amplification. Therefore, scalable systems should not simply move rights from humans to LLMs; they should move routine, checkable rights to deterministic components and reserve human attention for high-leverage exceptions.

Illustrative design implications

Autonomous anomaly management

Autonomous anomaly management illustrates the value of increasing LLM rights under scalability and adaptability pressure. Human-dependent anomaly-management pipelines can be too slow for complex systems, while rule-based approaches may struggle when system behavior evolves (Barenji and Khoshgoftar 2025). An LLM-enabled agent can interpret heterogeneous logs, sensor descriptions, and operational context, then propose explanations and responses. Yet full LLM control would be risky because anomaly response may affect real infrastructure. A rights-aware design would allocate information synthesis and hypothesis generation partly to the LLM, anomaly thresholds and policy constraints to deterministic components, and high-impact response approval to humans. As the system matures, repeated expert responses could be codified into deterministic playbooks, gradually shifting recurring rights away from humans without granting unconstrained authority to the LLM.

Workplace task automation

The workplace-agent evidence in Z. Z. Wang et al. (2025) suggests that current agents can be fast and inexpensive but still flawed. A rights-aware interpretation is that agents should not automatically receive end-to-end task rights merely because they can operate software tools. Instead, programmable subtasks with clear verification can be delegated to LLM or deterministic components, while visually grounded, ambiguous, or quality-sensitive steps should retain human validation. This supports augmentation over full automation when the LLM's process is efficient but its output quality remains uncertain. It also suggests that workflow demonstrations and induced workflows can be used to identify which decision nodes are safe to delegate.

Education and feedback-sensitive work

Educational AI illustrates the importance of human validation and contextual judgment. AI grading or feedback tools can improve efficiency, but rigid answer-key matching and automated feedback can miss contextually correct student responses (Yu et al. 2025). A rights-aware design would give the LLM drafting rights for feedback, deterministic components rights over rubric structure and consistency checks, instructors validation rights over grades or sensitive interventions, and students or end users contestation rights when feedback seems inappropriate. Over time, instructor corrections and student feedback can be converted into evaluation rules or examples, shifting some repeated judgments into deterministic or retrieval-based guidance while preserving human authority over ambiguous cases.

Domain-expert visualization and engineering workflows

In engineering or visualization tasks, RAG alone may retrieve relevant information but fail to produce expert-quality outputs. Uulu et al. (2026) argues that codified domain-expert knowledge can improve visualization generation beyond RAG-only code generation. Under the decision-rights view, the key mechanism is that expert codification shifts rights over quality criteria, chart selection, and domain appropriateness away from the LLM's generic judgment and into reusable protocols. The LLM still performs natural-language interpretation and code generation, but deterministic or expert-authored guidance controls what counts as an acceptable solution.

Model routing and cost-sensitive orchestration

Model routing illustrates that decision rights also apply to component selection. A system may use a small model for routine classification, a stronger model for difficult reasoning, a deterministic calculator for arithmetic, a retrieval pipeline for evidence, and a human reviewer for sensitive exceptions. Routing research shows that model selection can be optimized for quality, latency, cost, and regret (Sikeridis, Ramdass, and Pareek 2024; Tsiourvas, Sun, and Perakis 2025; Lai and Ye 2026). A rights-aware interpretation is that the router holds action-selection rights over which component will decide next. Therefore, router governance matters: a poor router can silently give weak models high-stakes reasoning rights or overuse expensive models in cases where deterministic checks would suffice.

Discussion

Decision-rights allocation helps explain why apparently similar agentic systems differ in reliability and governance. Two systems may both use RAG, memory, and tool calling, but one may let the LLM control query generation, source selection, memory writing, and action execution, while another uses curated retrieval, validated memory, schema-constrained tool calls, and human approval. Component labels alone cannot capture this difference. Decision-rights allocation can.

The perspective also helps unify several debates. The debate between rules and LLMs becomes a question of when deterministic rights should constrain stochastic reasoning. The debate between automation and augmentation becomes a question of which workflow nodes should retain human rights. The debate about autonomy becomes a question of which rights are transferred away from humans, not simply how independently the system operates. The debate about explainability becomes a question of whether humans and deterministic systems have enough information to exercise review and correction rights. The debate about agent memory becomes a question of who controls persistence and reuse.

This paper does not claim that decision rights are easy to measure. Effective rights may differ from formal rights. A human may formally approve an output but lack the information needed to challenge it. A deterministic validator may formally constrain action format but fail to constrain action meaning. An LLM may formally only draft recommendations but effectively determine the decision by framing the options. Future work should therefore develop empirical methods for estimating effective rights from system logs, interface design, approval behavior, routing records, memory-write events, user feedback, and failure cases. Agent evaluation should also move beyond final-output scoring toward trajectory, workspace, artifact, safety, robustness, cost, and long-horizon assessment (Yehudai et al. 2025; Zhuge et al. 2024).

Another limitation is that the three-party model is intentionally coarse. It treats all humans as one party, but organizations contain users, developers, managers, auditors, and affected communities with different interests. It treats deterministic components as one party, but rule engines, retrievers, validators, and routers can conflict. It treats stochastic LLMs as one party, but multi-agent systems may contain multiple models with different roles. The coarse model is useful for a first draft because it clarifies the main trade-off, but later versions should extend it to multi-stakeholder and multi-agent settings.

A further limitation is the boundary of the LLM component. In open-source deployments, designers may inspect prompts, retrieval, decoding settings, validators, and model versions. In API-based deployments, some provider-side safety filters, retrieval, or tool-use mechanisms may be hidden. The framework handles this by distinguishing explicit rights from observed reliability: hidden provider mechanisms may make a service safer, but they do not give the deploying organization explicit deterministic control rights unless they are exposed through inspectable interfaces. This boundary issue should become an empirical research topic because many organizations will govern agentic systems through third-party model services rather than fully transparent model stacks.

Conclusion

LLM-enabled agentic systems are not only collections of models, tools, memories, and workflows. They are arrangements of decision rights. Every design choice determines whether humans, deterministic components, or stochastic LLMs have authority over goals, evidence, reasoning, actions, validation, and memory. This paper proposed decision-rights allocation as a compact internal-mechanism perspective for comparing and designing such systems. It showed how common design patterns can be interpreted as rights-shifting mechanisms and proposed a simple model for choosing the desirable allocation based on scalability, adaptability, risk, auditability, codifiability, model reliability, verification availability, human capacity, and the cost of errors at scale.

The main practical implication is that agentic-system design should not begin with the question, "How autonomous can the agent be?" It should begin with the question, "Which party should hold which decision rights at which workflow step?" LLMs should receive rights where semantic flexibility and adaptation create value; deterministic components should receive rights where rules, schemas, retrieval, validators, provenance, and routing improve reliability; humans should retain rights where judgment, accountability, feedback, and legitimacy matter. A theory of LLM-enabled agentic systems can then be built not around autonomy as a single aspiration, but around explicit, inspectable, and optimizable allocation of decision rights.

References

Amini, Soufiane, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, et al. 2025. "Open Agent Specification (Agent Spec): A Unified Representation for AI Agents." https://arxiv.org/abs/2510.04173v4.

Barenji, Reza Vatankhah, and Sina Khoshgoftar. 2025. "Agentic AI for Autonomous Anomaly Management in Complex Systems." https://arxiv.org/abs/2507.15676v1.

Cao, Sheng, Zhao Chang, Chang Li, Hannan Li, Liyao Fu, and Ji Tang. 2026. "The Auton Agentic AI Framework." https://arxiv.org/abs/2602.23720v1.

Cheng, Edward, and Jeshua Cheng. 2026. "A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows." https://arxiv.org/abs/2604.23049v1.

Cihon, Peter, Merlin Stein, Gagan Bansal, Sam Manning, and Kevin Xu. 2025. "Measuring AI Agent Autonomy: Towards a Scalable Approach with Code Inspection." https://arxiv.org/abs/2502.15212v1.

Dumas, Marlon, Fredrik Milani, and David Chapela-Campa. 2026. "Agentic Business Process Management Systems." https://arxiv.org/abs/2601.18833v1.

Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. "Levels of Autonomy for AI Agents." https://arxiv.org/abs/2506.12469v2.

Fiorini, Sandro Rama, Leonardo G. Azevedo, Raphael M. Thiago, Valesca M. de Sousa, Anton B. Labate, and Viviane Torres da Silva. 2025. "Episodic Memory in Agentic Frameworks: Suggesting Next Tasks." https://arxiv.org/abs/2511.17775v1.

Garrido-Merchán, Eduardo C., and Cristina Puente. 2025. "GOFAI Meets Generative AI: Development of Expert Systems by Means of Large Language Models." https://arxiv.org/abs/2507.13550v2.

Gumaan, Esmail. 2025. "Theoretical Foundations and Mitigation of Hallucination in Large Language Models." https://arxiv.org/abs/2507.22915v1.

Händler, Thorsten. 2023. "Balancing Autonomy and Alignment: A Multi-Dimensional Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures." https://arxiv.org/abs/2310.03659v1.

Hariharan, Mohanakrishnan. 2025. "Reinforcement Learning Integrated Agentic RAG for Software Test Cases Authoring." https://arxiv.org/abs/2512.06060v1.

Hou, Wanda, Leon Zhou, Hong-Ye Hu, Yubei Chen, Yi-Zhuang You, and Xiao-Liang Qi. 2025. "How Focused Are LLMs? A Quantitative Study via Repetitive Deterministic Prediction Tasks." https://arxiv.org/abs/2511.00763v2.

Hu, Shengran, Cong Lu, and Jeff Clune. 2024. "Automated Design of Agentic Systems." https://arxiv.org/abs/2408.08435v2.

Joshi, Tanmay, Shourya Aggarwal, Anusa Saha, Aadi Pandey, Shreyash Dhoot, Vighnesh Rai, Raxit Goswami, Aman Chadha, Vinija Jain, and Amitava Das. 2026. "Stochastic CHAOS: Why Deterministic Inference Kills, and Distributional Variability Is the Heartbeat of Artifical Cognition." https://arxiv.org/abs/2601.07239v1.

Kamalov, Firuz, David Santandreu Calonge, Linda Smail, Dilshod Azizov, Dimple R. Thadani, Theresa Kwong, and Amara Atif. 2025. "Evolution of AI in Education: Agentic Workflows." https://arxiv.org/abs/2504.20082v2.

Kandikatla, Laxmiraju, and Branislav Radeljic. 2025. "AI and Human Oversight: A Risk-Based Framework for Alignment." https://arxiv.org/abs/2510.09090v1.

Kiprono, Moses. 2025. "Mathematical Analysis of Hallucination Dynamics in Large Language Models: Uncertainty Quantification, Advanced Decoding, and Principled Mitigation." https://arxiv.org/abs/2511.15005v1.

Klissarov, Martin, Jonathan Cook, Diego Antognini, Hao Sun, Jingling Li, Natasha Jaques, Claudiu Musat, and Edward Grefenstette. 2026. "Improving Interactive in-Context Learning from Natural Language Feedback." https://arxiv.org/abs/2602.16066v1.

Kubam, Chandra Sekhar. 2025. "Agentic AI for Autonomous, Explainable, and Real-Time Credit Risk Decision-Making." https://doi.org/10.5281/zenodo.18008872.

Lai, Guannan, and Han-Jia Ye. 2026. "When Routing Collapses: On the Degenerate Convergence of LLM Routers." https://arxiv.org/abs/2602.03478v1.

Laird, John E. 2022. "Introduction to Soar." https://arxiv.org/abs/2205.03854v1.

Lamo Castrillo, Victor de, Habtom Kahsay Gidey, Alexander Lenz, and Alois Knoll. 2025. "Fundamentals of Building Autonomous LLM Agents." https://arxiv.org/abs/2510.09244v1.

Lazer, Sahaya Jestus, Kshitiz Aryal, Maanak Gupta, and Elisa Bertino. 2026. "A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-Case Prototypes." https://arxiv.org/abs/2601.05293v1.

Li, Wenhao, Bo Jin, Mingyi Hong, Changhong Lu, and Xiangfeng Wang. 2025. "Optimization Problem Solving Can Transition to Evolutionary Agentic Workflows." https://arxiv.org/abs/2505.04354v1.

Li, Yuanzhe, Jianing Deng, Jingtong Hu, Tianlong Chen, Song Wang, and Huanrui Yang. 2026. "Dynamic Mix Precision Routing for Efficient Multi-Step LLM Interaction." https://arxiv.org/abs/2602.02711v1.

McGee, Liam, James Harvey, Lucy Cull, Andreas Vermeulen, Bart-Floris Visscher, and Malvika Sharan. 2025. "Enabling Ethical AI: A Case Study in Using Ontological Context for Justified Agentic AI Decisions." https://arxiv.org/abs/2512.04822v1.

Milosevic, Zoran, and Fethi Rabhi. 2026. "Architecting Agentic Communities Using Design Patterns." https://arxiv.org/abs/2601.03624v2.

Naqvi, Saba, Mohammad Baqar, and Nawaz Ali Mohammad. 2026. "The Rise of Agentic Testing: Multi-Agent Systems for Robust Software Quality Assurance." https://arxiv.org/abs/2601.02454v1.

Pellejero, Jorge, Luis A. Hernández Gómez, Luis Mendo Tomás, and Zoraida Frias Barroso. 2025. "Agentic AI for Mobile Network RAN Management and Optimization." https://arxiv.org/abs/2511.02532v1.

Pittman, Jason M. 2024. "A Measure for Level of Autonomy Based on Observable System Behavior." https://arxiv.org/abs/2407.14975v1.

Qian, Cheng, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jielin Qiu, Haolin Chen, Zhiwei Liu, et al. 2025. "xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning." https://arxiv.org/abs/2510.08439v1.

Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. "AI Agents Vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges." https://doi.org/10.1016/j.inffus.2025.103599.

Sikeridis, Dimitrios, Dennis Ramdass, and Pranay Pareek. 2024. "PickLLM: Context-Aware RL-Assisted Large Language Model Routing." https://arxiv.org/abs/2412.12170v1.

Singh, Yash Raj. 2025. "Graph-Memoized Reasoning: Foundations Structured Workflow Reuse in Intelligent Systems." https://arxiv.org/abs/2511.15715v1.

Son, Tong Duy, Zhihao Liu, Piero Brigida, Yerlan Akhmetov, Gurudevan Devarajan, Kai Liu, and Ajinkya Bhave. 2026. "Automotive Engineering-Centric Agentic AI Workflow Framework." https://arxiv.org/abs/2604.07784v1.

Taparia, Aditya, Ransalu Senanayake, Kowshik Thopalli, and Vivek Narayanaswamy. 2026. "The Anatomy of Uncertainty in LLMs." https://arxiv.org/abs/2603.24967v1.

Tsiourvas, Asterios, Wei Sun, and Georgia Perakis. 2025. "Causal LLM Routing: End-to-End Regret Minimization from Observational Data." https://arxiv.org/abs/2505.16037v2.

Uulu, Choro Ulan, Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, and Helena Holmström Olsson. 2026. "How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework." https://arxiv.org/abs/2601.15153v1.

Vakharia, Priyesh, Abigail Kufeldt, Max Meyers, Ian Lane, and Leilani Gilpin. 2024. "ProSLM : A Prolog Synergized Language Model for Explainable Domain Specific Knowledge Based Question Answering." https://doi.org/10.1007/978-3-031-71170-1_23.

Vsevolodovna, Ruslan Idelfonso Magana, and Marco Monti. 2025. "Enhancing Large Language Models Through Neuro-Symbolic Integration and Ontological Reasoning." https://arxiv.org/abs/2504.07640v2.

Wang, Jun. 2025. "Memento 2: Learning by Stateful Reflective Memory." https://arxiv.org/abs/2512.22716v3.

Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. 2023. "A Survey on Large Language Model Based Autonomous Agents." https://doi.org/10.1007/s11704-024-40231-1.

Wang, Zora Zhiruo, Yijia Shao, Omar Shaikh, Daniel Fried, Graham Neubig, and Diyi Yang. 2025. "How Do AI Agents Do Human Work? Comparing AI and Human Workflows Across Diverse Occupations." https://arxiv.org/abs/2510.22780v2.

Wissuchek, Christopher, and Patrick Zschech. 2025. "Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework." https://arxiv.org/abs/2508.00844v1.

Wray, Robert E., James R. Kirk, and John E. Laird. 2025. "Applying Cognitive Design Patterns to General LLM Agents." https://arxiv.org/abs/2505.07087v2.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al.

2023. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” https://arxiv.org/abs/2308.08155v2.

Yamaguchi, Tomoaki, Yutong Zhou, Masahiro Ryo, and Keisuke Katsura. 2025. “Agentic Explainable Artificial Intelligence (Agentic XAI) Approach to Explore Better Explanation.” https://arxiv.org/abs/2512.21066v3.

Yehudai, Asaf, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. “Survey on Evaluation of LLM-Based Agents.” https://arxiv.org/abs/2503.16416v1.

Yu, Ning, Jie Zhang, Sandeep Mitra, Rebecca Smith, and Adam Rich. 2025. “AI-Educational Development Loop (AI-EDL): A Conceptual Framework to Bridge AI Capabilities with Classical Educational Theories.” https://arxiv.org/abs/2508.00970v2.

Zhang, Guangwei. 2025. “Knowledge Protocol Engineering: A New Paradigm for AI in Domain-Specific Knowledge Work.” https://arxiv.org/abs/2507.02760v1.

Zhang, Linghao. 2026. “Nurture-First Agent Development: Building Domain-Expert AI Agents Through Conversational Knowledge Crystallization.” https://arxiv.org/abs/2603.10808v1.

Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability.” https://arxiv.org/abs/2601.17168v1.

Zhu, Yizhang, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, et al. 2025. “A Survey of Data Agents: Emerging Paradigm or Overstated Hype?” https://arxiv.org/abs/2510.23587v2.

Zhuge, Mingchen, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, et al. 2024. “Agent-as-a-Judge: Evaluate Agents with Agents.” https://arxiv.org/abs/2410.10934v2.